Introducción al *Unified Modeling Language*

Ingeniería del Software I
Facultad de Matemática, Astronomía y Física
Universidad Nacional de Córdoba

Agosto de 2011

La Necesidad de Modelar

La complejidad asociada al desarrollo de software nos ha llevado a observar que:

- Concentrar las metodologías de desarrollo en etapas de implementación es en muchos casos contraproducente (e.g., pueden llevar a descubrir errores en los requisitos cuando ya se ha invertido mucho esfuerzo en su implementación).
- Es importante *analizar* el problema hasta tener una buena comprensión del mismo.
- Es importante contar con un *plan* para el desarrollo de una solución al problema, antes de comenzar a implementar la solución.

Los últimos dos puntos corresponden a las etapas de *análisis* y *diseño*. Para llevar adelante las tareas de análisis y diseño necesitamos poder *describir*, o *modelar*, diferentes aspectos del problema y de su solución.

El Modelado de Software

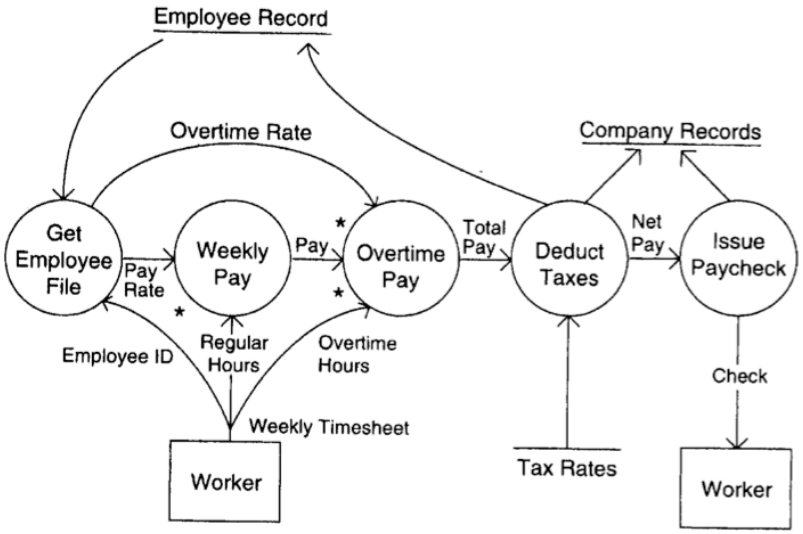
Modelar es describir algún aspecto del problema a resolver o de su solución. Esta actividad en general demanda:

- la interacción entre varios desarroladores (e.g., discutiendo ideas, decisiones de diseño, etc)
- la interacción entre desarroladores y clientes (e.g., para validar requisitos, etc)

Los lenguajes de modelado *estandarizan* la notación utilizada para describir modelos, para facilitar las actividades mencionadas. Más aún, algunas tareas de análisis pueden *automatizarse*, por lo cual es necesario definir de manera precisa los lenguajes de modelado a utilizar.

Modelado en el Proceso de Desarrollo Estructurado

Al igual que en los lenguajes procedimentales, la metodología de desarrollo estructurado se concentran en el uso de la *abstracción procedimental* como mecanismo para lidiar con la complejidad de modelado/desarrollo, y la división o separación de tareas en subtareas.
La notación central en esta metodología es la brindada por los *diagramas de flujo de datos*.



La descripción de abstracciones de datos es, en algún sentido, auxiliar, y se da a través de *diccionarios de datos*.

Modelado en el Desarrollo Orientado a Objetos

Es claro que en el modelado orientado a objetos es *importante* centrar las actividades de modelado en la *abstracción de datos*. El proceso estructurado es entonces inadecuado para esta tarea.
Desde las primeras notaciones para el modelado orientado a objetos se incorpora notación para la descripción de *abstracciones de datos* (e.g., en las tarjetas CRC).
Varias metodologías de desarrollo orientado a objetos fueron propuestas, entre las que se destacan:

- Diseño por Contratos
- OMT (Object Modeling Technique)
- El Método Booch
- OOSE (Object Oriented Software Engineering)

Cada una de estas metodologías ofrece su propia notación. UML (Unified Modeling Language) surgió como un intento de integrar y estandarizar las tres últimas.

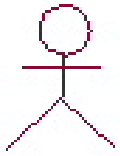
UML

UML es una *familia* de notaciones de utilidad para el desarrollo orientado a objetos, que surgió a partir de una iniciativa de integrar algunas metodologías de desarrollo orientado a objetos y sus notaciones. Algunas de las notaciones involucradas en UML son las siguientes:

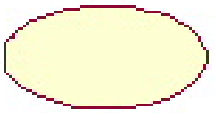
- Diagramas de Casos de Uso
- Diagramas de Clases
- Diagramas de Actividades
- Diagramas de Componentes
- Diagramas de Colaboración
- Diagramas de Estados
- Diagramas de Secuencias

Diagramas de Casos de Uso

Los diagramas de casos de uso son utilizados en etapas tempranas del proceso de desarrollo para describir *casos de uso*, es decir, conjuntos de escenarios de uso del sistema.
Los diagramas de casos de uso se utilizan para describir las funcionalidades de un sistema, y los *actores* involucrados en estas funcionalidades.
Los elementos fundamentales de la notación son los *actores* y los *casos de uso* (o servicios)



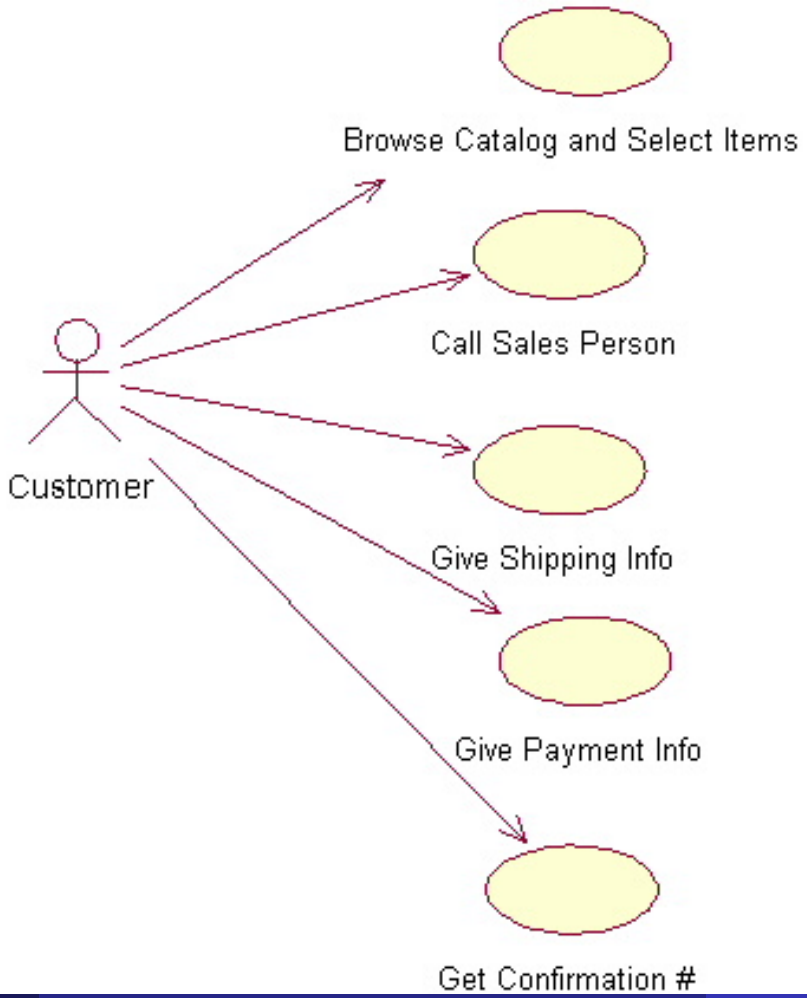
Actor



Use Case

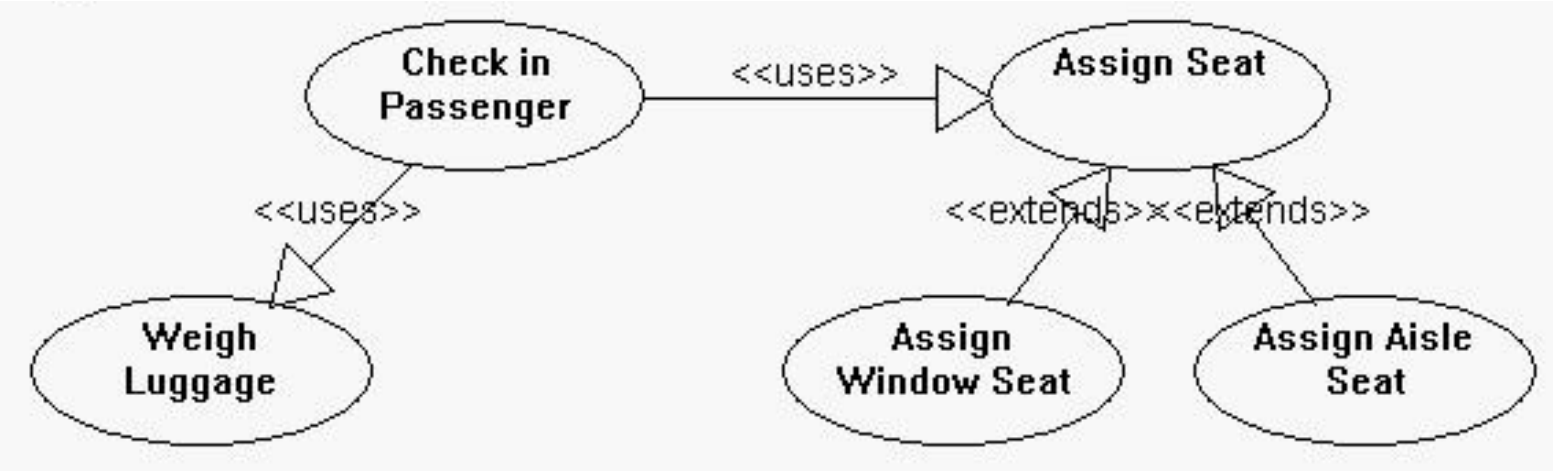
Diagramas de Casos de Uso (cont.)

Los casos de uso *no* describen secuenciación de acciones, sino funcionalidades de alto nivel del sistema.



Diagramas de Casos de Uso (cont.)

Además de casos de uso y actores, los diagramas de casos de uso constan de *relaciones entre casos de uso* (uso, extensión) y una representación de los elementos internos al sistema a desarrollar.

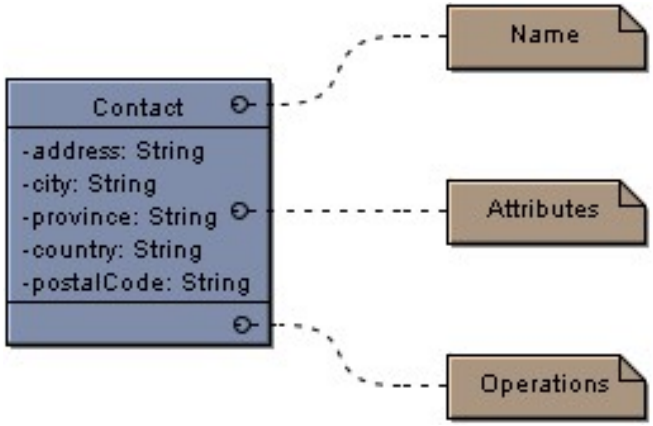


Diagramas de Clases

Los diagramas de clases describen los tipos de objetos en un sistema orientado a objetos, y las relaciones (estáticas) entre éstos. Estos diagramas se utilizan tanto en etapas de análisis (donde también pueden aparecer entidades que son parte de la descripción del problema pero no del sistema en sí) como en etapas de diseño.

Los diagramas de clases describen de forma gráfica diseños orientados a objetos.

Los elementos fundamentales de la notación son las *clases* (y los elementos que las constituyen, como su nombre, atributos y métodos) y las *asociaciones* (relaciones entre clases)

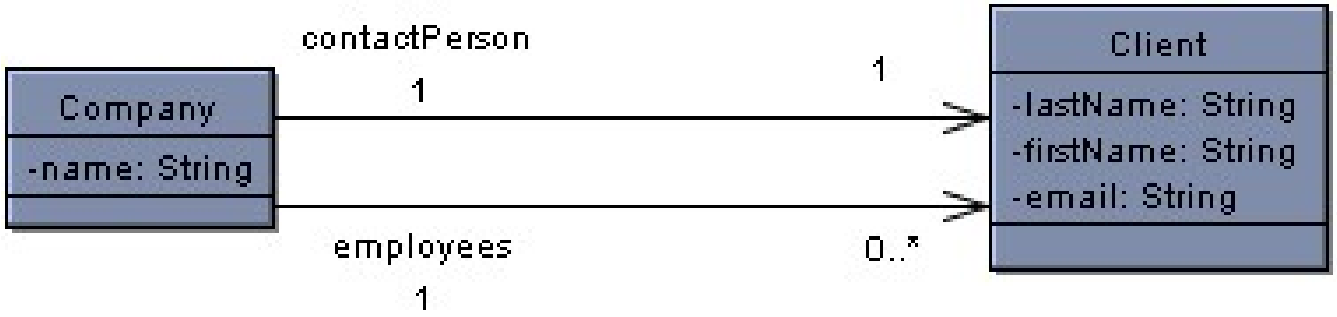


Diagramas de Clases (cont.)

Las asociaciones permiten describir relaciones entre clases (similar a la finalidad de las relaciones en los diagramas de entidad/relación en Bases de Datos)

Las asociaciones pueden acompañarse de restricciones en forma de *multiplicidades*

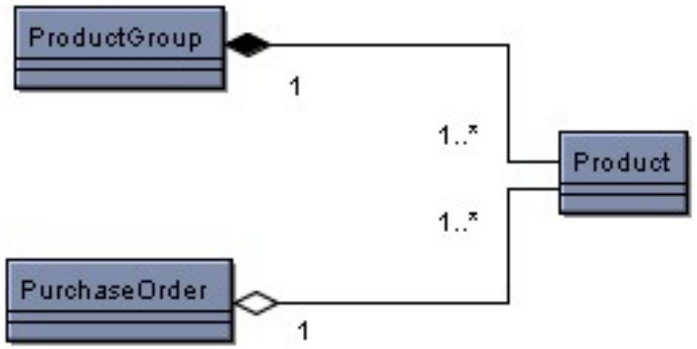
> Éstas restringen el número de instancias que pueden estar relacionadas en el sistema en ejecución



Diagramas de Clases (cont.)

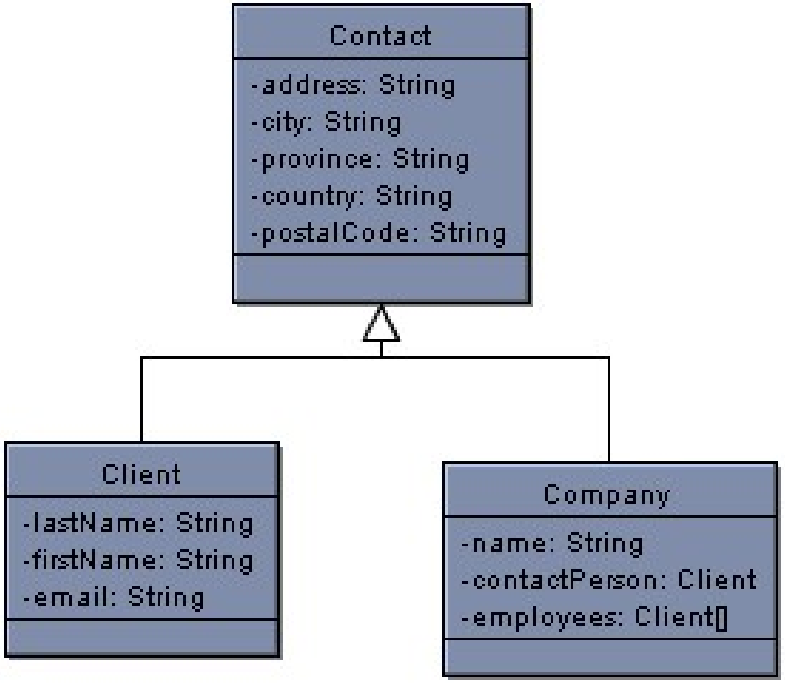
Existen algunos tipos particulares de asociaciones, que tienen notación específica: las *agregaciones* y las *composiciones*

- Ambas corresponden a la noción intuitiva de la relación "es parte de" entre entidades
- La *composición* es más fuerte: si se destruye el objeto "compuesto", se destruyen sus componentes (se denota con un rombo relleno en lugar de una punta de flecha)
- La *agregación* es más débil: que se destruya el objeto "compuesto" no implica que se destruyan sus componentes (se denota con un rombo vacío en lugar de una punta de flecha)



Diagramas de Clases (cont.)

Una de las relaciones más importantes en orientación a objetos es la *herencia*. Ésta posee su representación en los diagramas de clases, que es similar a la utilizada en diagramas de entidad/relación en bases de datos



La herencia *no* se limita a herencia simple en UML. Las clases abstractas, usadas generalmente en combinación con herencia, se denotan con el nombre en cursiva.

Diagramas de Estados

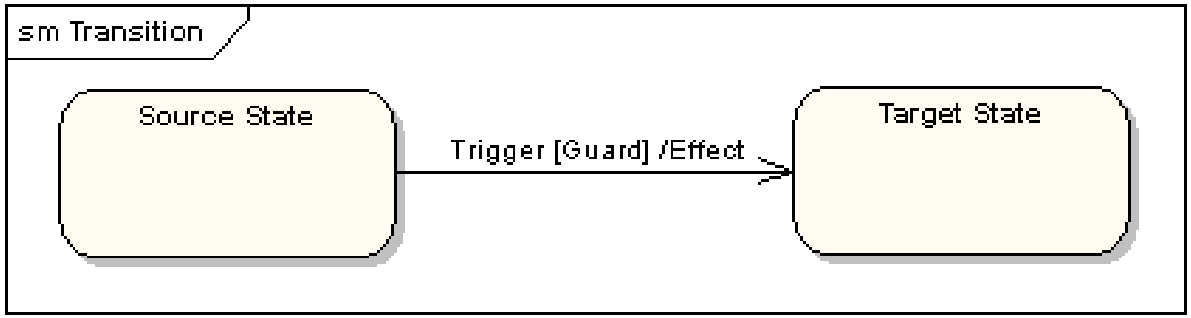
Los diagramas de estados permiten describir el comportamiento *dinámico* asociado a los objetos de una clase. Los diagramas de estados están basados en los *statecharts* de D. Harel.
Las características fundamentales de los diagramas de estados, tomadas de los statecharts, son:

- La posibilidad de organizar jerárquicamente los estados del sistema, mediante estados OR (estados *compuestos* en la nomenclatura de UML)
- La posibilidad de describir componentes *ortogonales* dentro del estado de los objetos, mediante estados AND (*regiones concurrentes* en la nomenclatura de UML)

Los elementos básicos que constituyen un diagrama de estados son los *estados* (OR y AND) y las *transiciones*.

Diagramas de Estados (cont.)

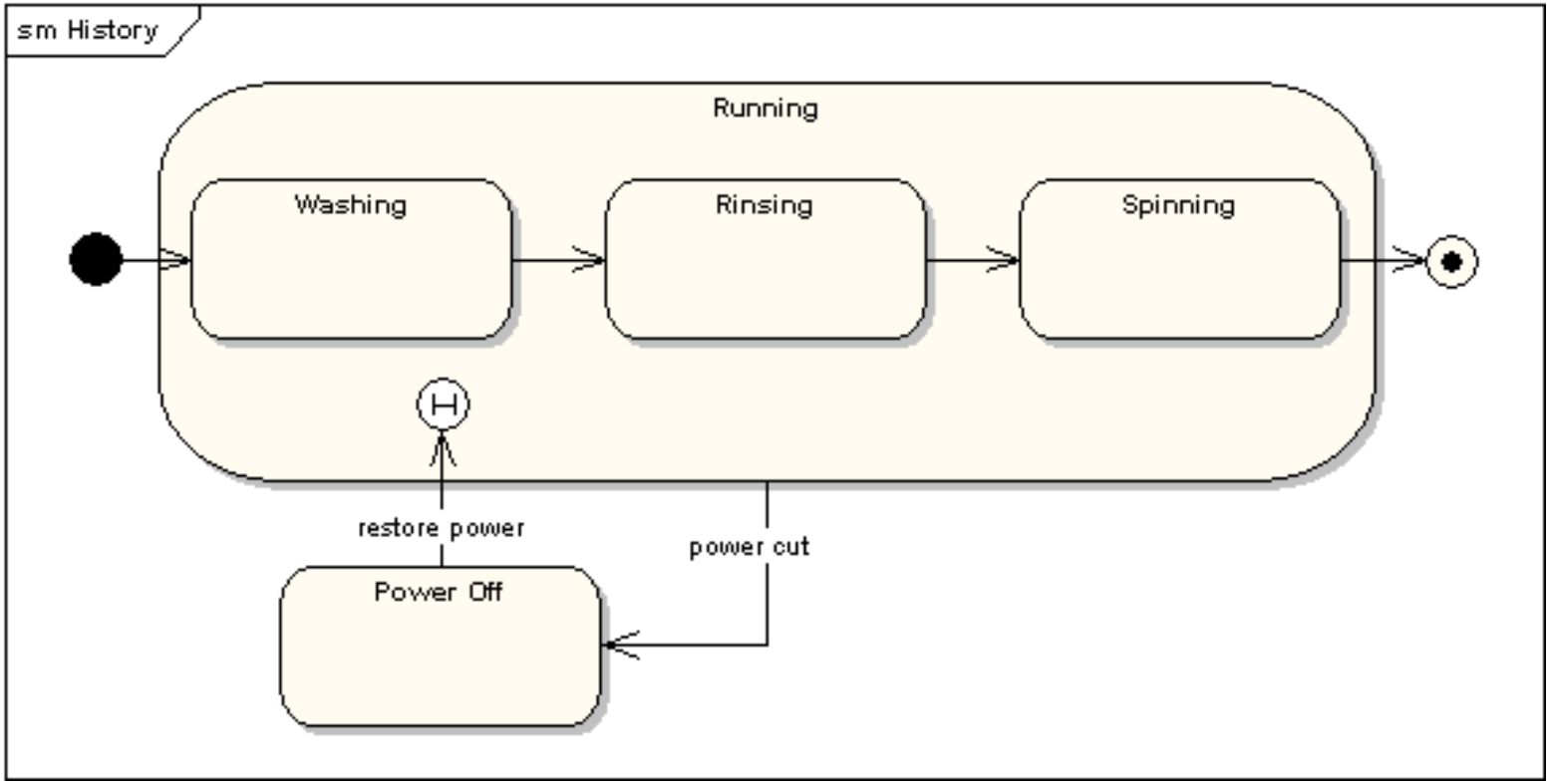
Los elementos que constituyen un diagrama de estados se describen graficamente de la siguiente manera:



Las transiciones poseen un evento de lanzamiento (trigger), pueden estar acompañados de guardas (condiciones adicionales para "disparar" la transición), y generar a su vez nuevos eventos.

Diagramas de Estados (cont.)

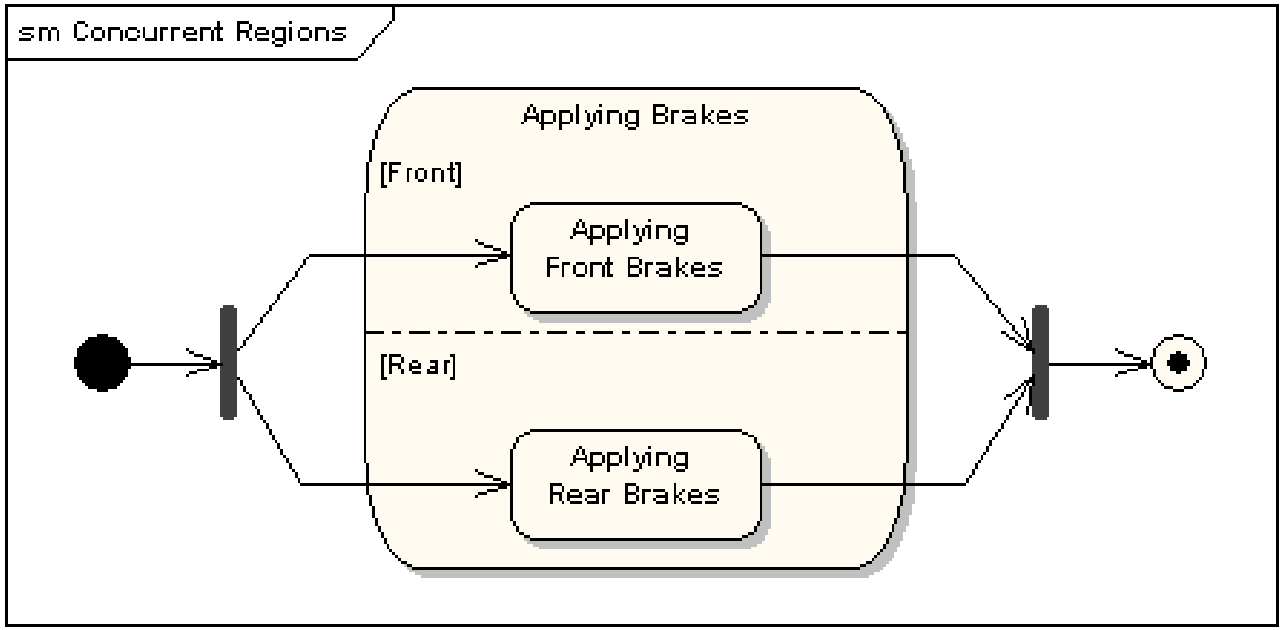
Los estados compuestos permiten incorporar detalle asociado a un estado, mediante transiciones entre "sub-estados" del estado compuesto:



En este tipo de descripciones suelen ser de utilidad los estados iniciales, finales, por defecto, y la *historia*.

Diagramas de Estados (cont.)

Las regiones concurrentes permiten describir de manera clara y concisa sistemas cuyo estado está compuesto por elementos ortogonales:



A diferencia de los diagramas de interacción, que describiremos a continuación, los diagramas de estados describen el comportamiento *total* del objeto especificado.

Diagramas de Interacción

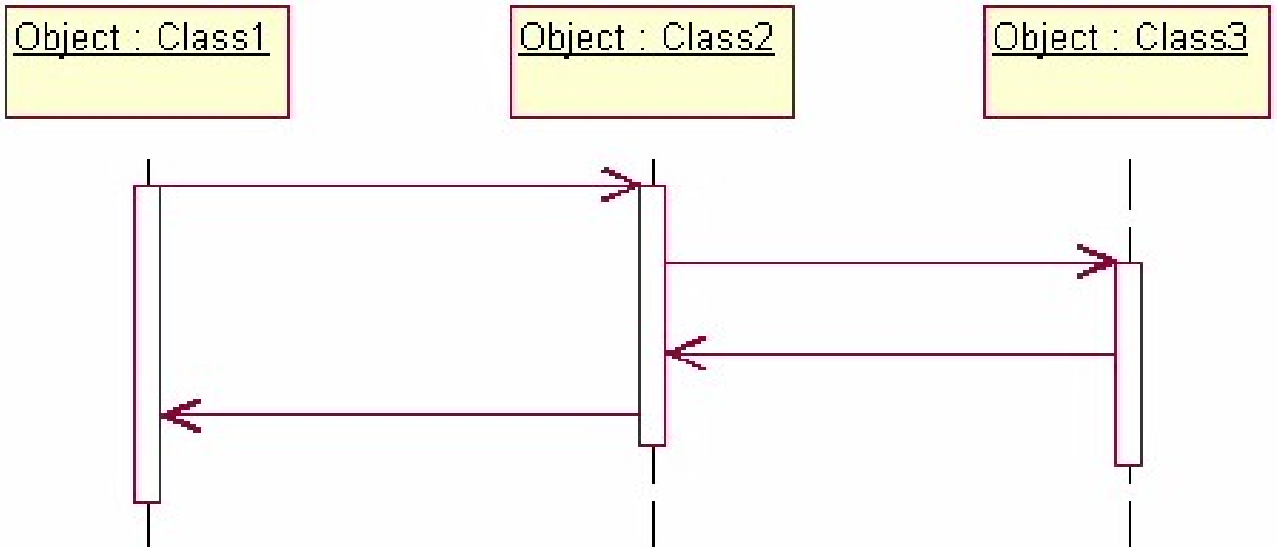
La finalidad de los diagramas de estado es describir el comportamiento total asociado con un objeto. Los *diagramas de interacción*, en cambio, tienen como finalidad describir *escenarios particulares* de ejecución del sistema.

Tenemos dos tipos de diagramas de interacción: los *diagramas de secuencias* y los *diagramas de colaboración*.

En ambos diagramas, los ingredientes fundamentales de la notación son las *instancias* (objetos) y los *mensajes*.

Diagramas de Secuencias

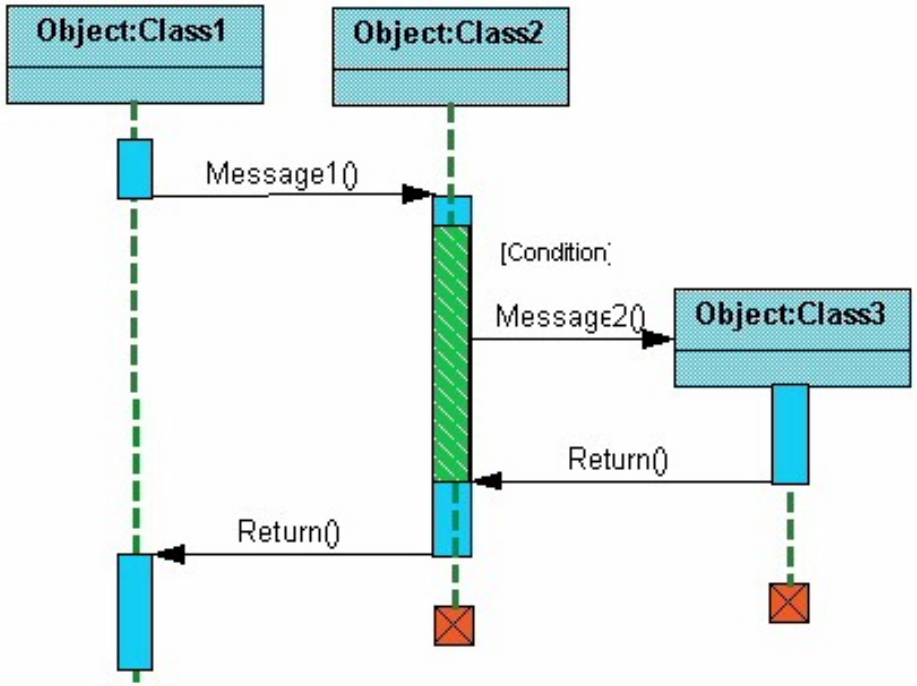
Los diagramas de secuencias tienen la siguiente forma:

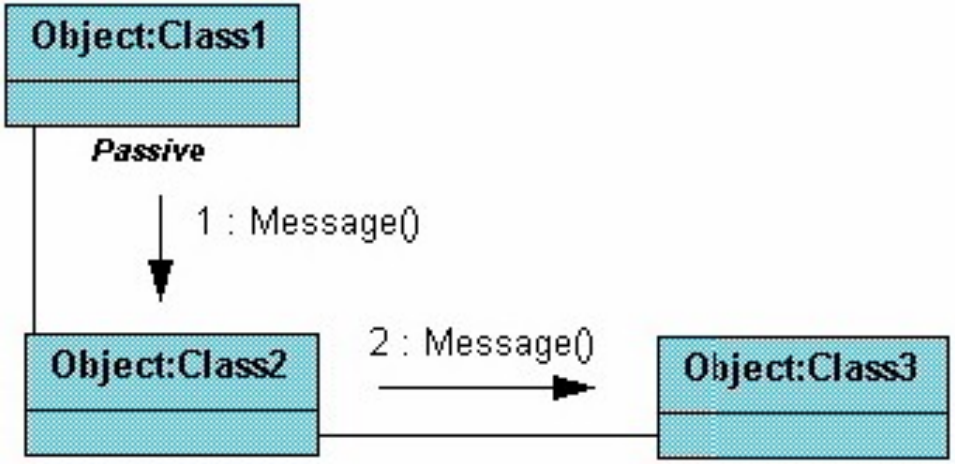


Los diagramas de secuencias describen un conjunto de objetos interactuando mediante el pasaje de mensajes (o invocaciones de servicios). La lectura de un diagrama de secuencias se realiza de izquierda a derecha, y de arriba hacia abajo.

Diagramas de Secuencias (cont.)

Las líneas debajo de la declaración de un objeto describe el tiempo de vida del objeto. Se puede describir (mediante rectángulos) la activación de objetos, y su destrucción (con una cruz dentro de un cuadrado).



Diagramas de Colaboración

Los diagramas de colaboración son similares a los diagramas de secuencias. En estos diagramas, la sintaxis está basada en la de los diagramas de clases (aunque los diagramas de colaboración describen situaciones *dinámicas*)